
Empirical Comparison of Convergence Behavior in Direct Feedback Alignment Compared to Backpropagation

Allie H. Zong

Department of Applied Mathematics
Harvard University
allie_zong@college.harvard.edu

Abstract

Backpropagation (BP) is the most commonly used learning rule in artificial neural networks, which allows for efficient computation of the gradients with which the weights and biases are updated in a network. In BP, the error signals are propagated backward through the network using the transposed forward weights. This leads to concerns about the biological plausibility of BP, and to the introduction of alternative learning rules that better resemble known neural circuitry. Direct Feedback Alignment (DFA) addresses this by replacing symmetric backward weights with fixed random feedback connections from the output units directly to each hidden layer. This work expands upon initial work in Nøkland (2016) by empirically comparing the convergence behavior of DFA and BP in deep, fully connected networks on MNIST. This is done by varying network depth, width, and training set size and subsequently examining convergence speed, gradient norms, and final test performance. The results show that DFA matches BP in final test error across a broad range of network architectures and data regimes, while maintaining robustness in deep networks with saturating nonlinearities where BP suffers from vanishing gradients. Thus, the findings in this paper reinforce DFA's capacity to perform at similar levels to BP across varying architectures and data regimes, making DFA a promising candidate for further exploration of biologically plausible learning rules.

1 Introduction

The recent successes of deep learning are largely attributed to the efficiency of the backpropagation algorithm (BP), which provides a way to compute the exact gradients in layered networks [1]. BP computes these gradients by propagating error signals backward through the network using the transpose of the forward weights. This requirement of symmetry between the forward and backward weights, a mechanism also known as the "weight transport problem", forms a large hindrance in reconciling BP with known biological constraints [6, 4]. BP is not a local learning rule, since the error signals from the output units have to be propagated backward through the network. As a result, BP is generally considered biologically implausible, spurring a new branch of research on biologically plausible alternative learning rules to BP that better align with current knowledge of neural circuitry [5].

The Direct Feedback Alignment (DFA) algorithm is an algorithm that offers a more biologically plausible learning rule compared to BP [2]. In DFA, the error signals are propagated backwards through the network using fixed, random feedback weights instead of the transposes of the forward weights in BP, thus removing the need for symmetry between the forward and backward weights.

Despite the randomly chosen fixed backward weights, DFA networks are effective in learning using these weights, showing that the network learns how to learn. DFA networks have been shown to train effectively in deep networks despite its approximations, though its convergence dynamics, particularly in different architectures and training regimes, remain much less understood than BP.

In DFA, learning still succeeds despite such crude feedback from the random weights. This is achieved due to the fact that the feedforward weights adapt so that the random feedback signals become aligned with the true gradient. In DFA, the output errors are also directly broadcasted to every hidden layer through the independent random matrices. Instead of recursively multiplying the weight transposes down each layer, each layer directly receives a global error signal that is locally transformed and combined with its own activation derivative. The resulting local error drives synaptic updates that depend on local activities: presynaptic activity, postsynaptic nonlinearity, and the modulatory error input. All in all, DFA eliminates the need for symmetric connectivity, a dedicated sequential backward pass, and allows all layers to be updated in parallel.

Figure 1 shows the novel architecture of DFA, which is explored in this work, compared to BP. This figure is directly taken from Nøkland (2016).

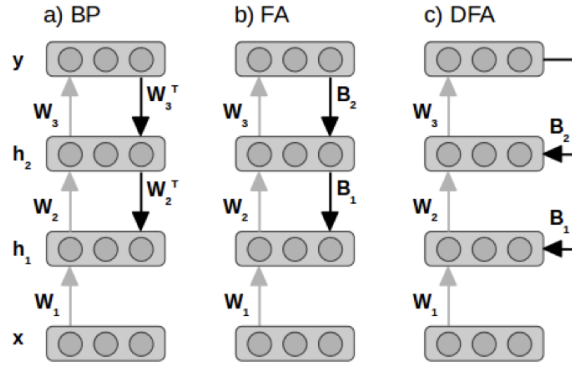


Figure 1: Schematic of different error propagation methods. Gray arrows indicate activation paths and black arrows indicate error paths. Weights that are adapted during learning as denoted as W_i , and weights that are fixed and random are denoted as B_i . a) Backpropagation. b) Feedback Alignment. c) Direct Feedback Alignment.

In this work, the convergence properties of DFA relative to BP are empirically investigated, with a focus on varying network architectures and data regimes. First, key results from Nøkland (2016) are reproduced and confirmed. Extending on prior work on shallow networks by Refinetti et al. (2021), this paper systematically explores how network depth and width affect convergence speed and test loss [3]. Additionally, data efficiency is examined by measuring stabilized test loss and convergence speed as a function of dataset size.

A notable result of the paper includes insights about how DFA bypasses the vanishing gradient issue in saturating functions such as the tanh and sigmoid activation functions. For BP, in deep networks, if layers saturate, this causes the error signal to the hidden layers to approach zero, effectively vanishing the gradient update for lower layers [7].

The findings in this paper highlight conditions under which DFA matches or diverges from BP, providing insight into scalability and advantages of DFA as a biologically plausible alternative to BP. These results contribute to the current ongoing efforts in neural computation to develop learning rules more similar to current knowledge of biological neural circuitry.

2 Methods

2.1 Direct Feedback Alignment (DFA)

Consider a feedforward network trained using DFA. For simplicity, assume the network only has 2 hidden layers. Any additional hidden layers follow the same update rules as the system outlined here.

Consider a feedforward network with input x , two hidden layers with activations h_1 and h_2 , and output $\hat{y}$. The forward pass is defined as

$$\begin{aligned} a_1 &= W_1 x + b_1, & h_1 &= f(a_1) \\ a_2 &= W_2 h_1 + b_2, & h_2 &= f(a_2) \\ a_y &= W_3 h_2 + b_3, & \hat{y} &= f_y(a_y) \end{aligned}$$

where f is the hidden-layer nonlinearity (tanh throughout this paper) and f_y is the output nonlinearity (sigmoid through this paper). Cross-entropy loss is used averaged across N points and summed over multiple outputs m :

$$J = -\frac{1}{N} \sum_{m,n} [y_{mn} \log \hat{y}_{mn} + (1 - y_{mn}) \log(1 - \hat{y}_{mn})]. \quad (1)$$

The error at the output layer is

$$e = \delta a_y = \frac{\partial J}{\partial a_y} = \hat{y} - y. \quad (2)$$

In standard BP, the error signals are propagated backward recursively:

$$\delta a_2 = (W_3^\top e) \odot f'(a_2), \quad (3)$$

$$\delta a_1 = (W_2^\top \delta a_2) \odot f'(a_1). \quad (4)$$

Thus, the weights are updates with $\Delta W_l \propto \delta a_l h_{l-1}^\top$ (with analogous bias updates giving $\Delta b_l \propto \delta a_l$).

DFA modifies this by transmitting the output error e directly to every hidden layer via fixed random matrices B_l :

$$\delta a_2 = (B_2 e) \odot f'(a_2), \quad (5)$$

$$\delta a_1 = (B_1 e) \odot f'(a_1). \quad (6)$$

The top layer still uses standard backpropagation ($\delta a_y = e$). The feedback matrices B_l are drawn once from $\mathcal{N}(0, 1/\sqrt{n_{\text{in}}})$ (where n_{in} is the input dimension to the layer) and remain frozen during training. When hidden layers have identical width, the same B can be reused.

BP and DFA were implemented in PyTorch, closely following the reference implementation. Pseudocode for the DFA training step can be found in Appendix A.

2.2 Experiments

Three sets of experiments were conducted on the MNIST dataset (60,000 training, 10,000 test examples).

Reproduction of baseline results. Fully-connected networks were trained matching architectures from Nøkland (2016) with tanh hidden unit activations and sigmoid output. Only unfrozen random feedback matrices were tested to verify that DFA achieves test performance comparable to BP.

Convergence across depth and width. A systematic grid search was performed over network depth (2 to 16 hidden layers) and width (200 to 1200 units per layer). For each configuration, BP and DFA were trained until convergence, which was defined as a decrease of less than 0.1 in test error percentage averaged over 5 epochs. Convergence speed was measured as number of epochs needed until convergence with means and standard deviations from 3 runs per architecture.

Data efficiency. To assess minimal data requirements, networks were trained on subsampled training sets (1k, 2k, 3k, 4k, 5k, 10k, 20k, 30k, 40k, 50k, and full 60k examples). Stabilized test loss after convergence as defined above was recorded as a function of dataset size for both BP and DFA, revealing regimes where the methods differ in sample efficiency.

All experiments used RMSprop optimization, batch size 256, BP learning rate of 0.0005, and DFA learning rate of 0.001. Identical initialization schemes were used for fair comparison where BP weights are initialized using Xavier random initialization, and BP biases, DFA weights and DFA biases were initialized to zero. Binary cross-entropy loss was used.

3 Results

Experimental results from Nøkland (2016) were first reproduced using fully-connected networks on MNIST with tanh activation. All MNIST inputs were scaled to be between 0 and 1. A 3x400 tanh network and a 2x800 tanh network were trained with both BP and DFA, see Figure 2. Networks trained with DFA show test error curves similar to BP, achieving comparable final test errors (2% for both methods), meaning DFA and BP showed similar capacities to learn and perform well on the test set. The hidden layer activations were visualized using t-SNE, see Figure 3 [8].

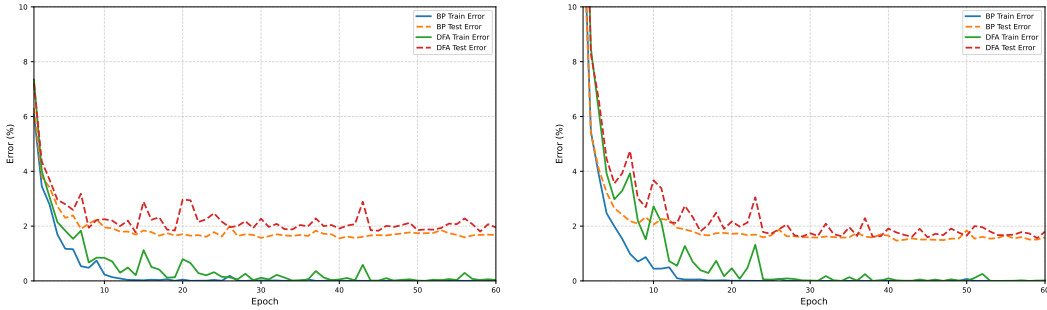


Figure 2: Left: Error curves for a 3x400 tanh network with both BP and DFA. Right: Error curves for a 2x800 tanh network with both BP and DFA.

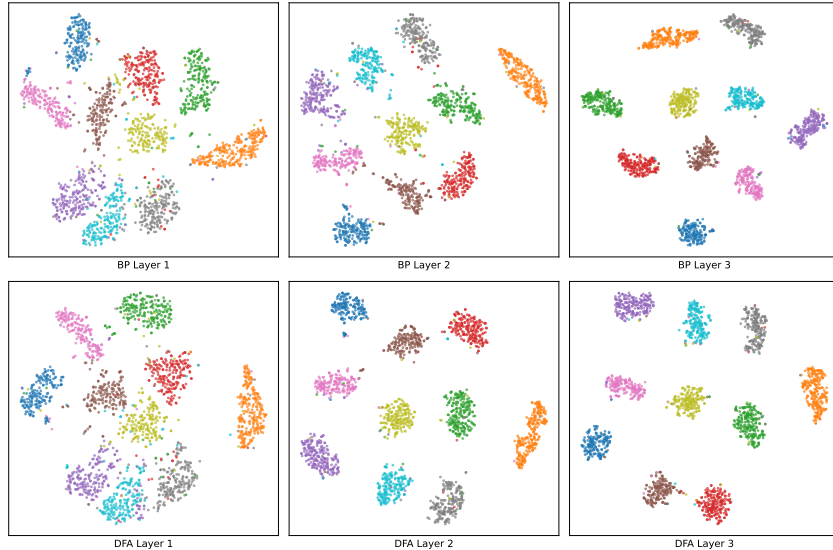


Figure 3: t-SNE visualization for a 3x400 tanh network. Top: 3 layers of hidden activations in BP. Bottom: 3 layers of hidden activations in DFA. From left to right: first hidden layer features, second layer features and third hidden layer features.

Both methods showed ability to learn features to discriminate between the 10 classes. At the third hidden layer, the clusters are clearly separated. The visible improvement between each layer indicates that both DFA and BP are continuously learning even in deeper hidden layers.

Further experiments were conducted on feedforward networks with varying architectures to compare DFA to BP for different widths and depths of networks. All networks were trained on MNIST for 60 epochs over 10 seeds, and mean and standard deviation are reported in Table 1, where a dash indicates that a run did not converge. DFA performs similarly to BP up to 10 layers, but at 30 layers, BP fails to converge due to the vanishing gradient problem [9]. This behavior aligns with known BP limitations in deep networks, especially with saturating activation functions like tanh and sigmoid. Using a ReLU activation function helps solve this vanishing gradient problem, but has its own drawbacks.

Table 1: Multi-seed performance comparison

Model	BP Mean	BP Std	DFA Mean	DFA Std
1x800	1.581	0.050	1.624	0.039
2x800	1.539	0.064	1.715	0.093
3x800	1.658	0.056	1.847	0.092
4x800	1.795	0.086	2.009	0.222
7x240	2.092	0.135	2.285	0.142
10x800	2.159	0.139	2.287	0.141
30x800	—	—	2.820	0.000

Since each hidden layer i in a DFA network gets direct broadcasting of the output error signal through a random matrix B_i , DFA manages to bypass the vanishing gradient problem due to its local properties. To verify DFA’s robustness to vanishing gradients, the gradient norms across hidden layers for the 30x800 tanh network were plotted during training in Figure 4. BP norms decay exponentially in deeper layers due to tanh saturation, while DFA maintains consistent norms via direct feedback, confirming theoretical claims in Nøklund (2016).

Convergence behavior of DFA was investigated relative to BP across varying network depths (2 to 16 layers) and widths (200 to 1200 units), averaged over 5 seeds per configuration. Convergence is defined as the number of epochs to reach a less than 0.1% difference in average test error across the last 5 epochs window compared to the previous 5.

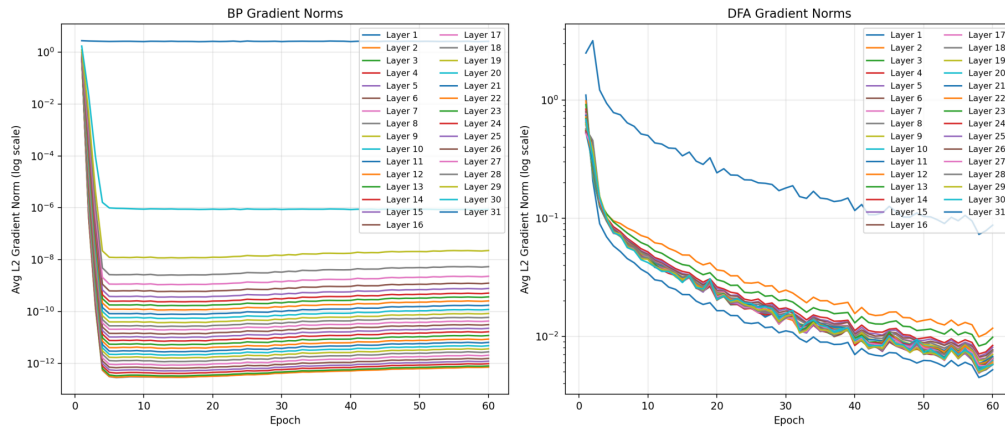


Figure 4: Left: Gradient norms of a 30x800 tanh network with BP over 60 epochs. Right: Gradient norms of a 30x800 tanh network with DFA over 60 epochs. Vanishing gradients are observed for BP after around 5 epochs, but DFA shows robustness against the vanishing gradient problem.

Figure 5 plots convergence epochs versus widths for fixed depths. At shallow depths (<8), BP seems to converge slightly faster than DFA, but for deeper networks, this trend becomes less apparent. The high uncertainty in convergence speed also indicates a high variance in convergence behavior, likely due to random weight initialization. From this experiment, convergence speed does not depend much on either depth or width in the investigated network architectures. Potential causes are the simplicity of the MNIST dataset, lack of more depth in the networks investigated, and potential convergence at local minima.

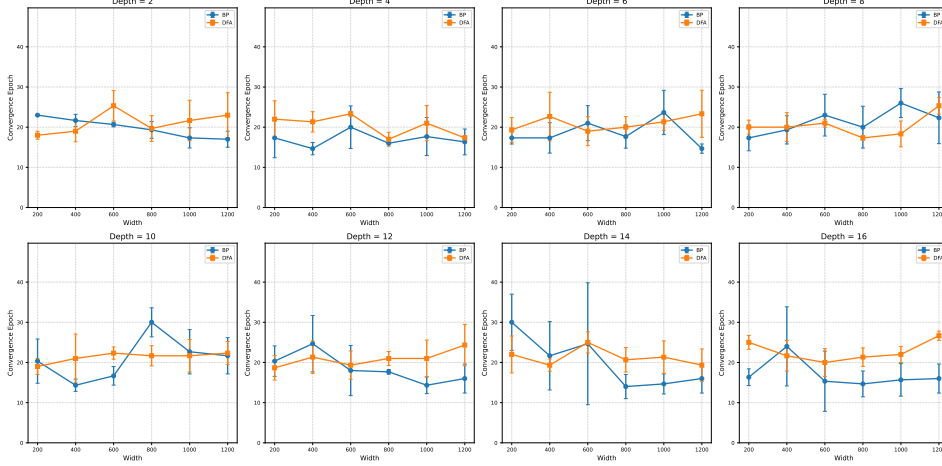


Figure 5: Convergence speed across varying widths while keeping depths constant.

To determine true performance of the converged networks, Figure 6 displays final test error percentage versus width keeping depths fixed. Starting at a depth of 10 layers, BP starts exhibiting vanishing gradients, where BP fails to find an effective minimum and instead converges due to lack of improvement in test error after gradients have vanished over epochs. One key observation is that width affects vanishing gradient in BP. The wider the network, the less deep the network needs to be to experience vanishing gradients. This trend adds a width element to the vanishing gradient problem.

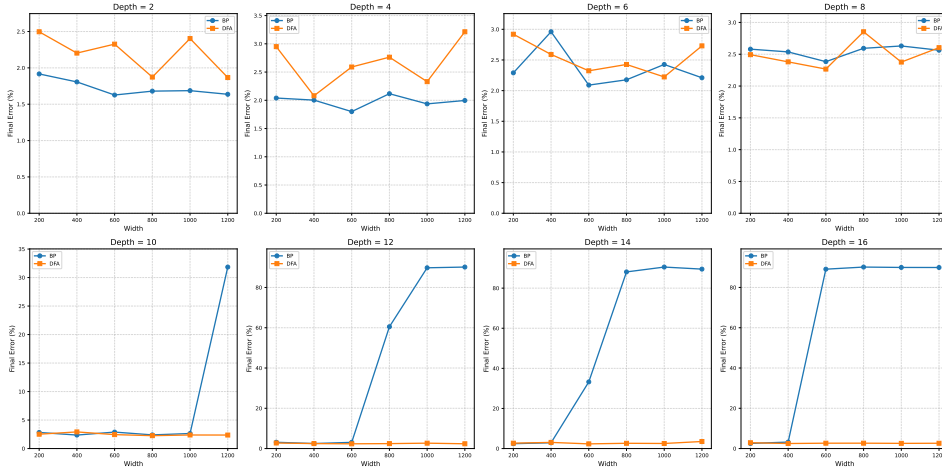


Figure 6: Final test error percentage across varying widths keeping depths constant. Showcases the vanishing gradient problem for deeper networks, where BP fails to converge at an effective minimum.

Subsequently, DFA and BP performance was examined on subsampled MNIST training sets (1k to 60k examples) averaged over 5 seeds, to assess minimal data requirements for convergence and optimal test performance on a 10x800 tanh network, see Figure 7. At 1k samples, erratic test loss was seen in BP, and DFA exhibited a two-phase learning behavior. The two-phase behavior of DFA is still

seen in 5k samples, where the test error first stalls at 90%, but rapidly drops to optimal convergence after a threshold epoch is reached. At 30k samples, this two-phase behavior disappeared in DFA, and the error curves were comparable for both BP and DFA to the full MNIST training set size of 60k. A likely cause of the two-phase behavior is that the feedforward weights must adapt to align with the fixed backward weights. Once this is achieved, there is a sharp decline in test error.

Figure 8 summarizes the training and convergence dynamics of the 10x800 tanh network under varying data regimes. Figure 8a shows that using more data in training helps with final test performance, with final test error percentage plateauing at around 20k data points, indicating that only about 1/3 of the dataset size is needed to achieve similar results as training with the full dataset. Figure 8b shows that using more data helps BP converge faster, but DFA shows a peak in convergence epochs and then declines again after 5k samples.

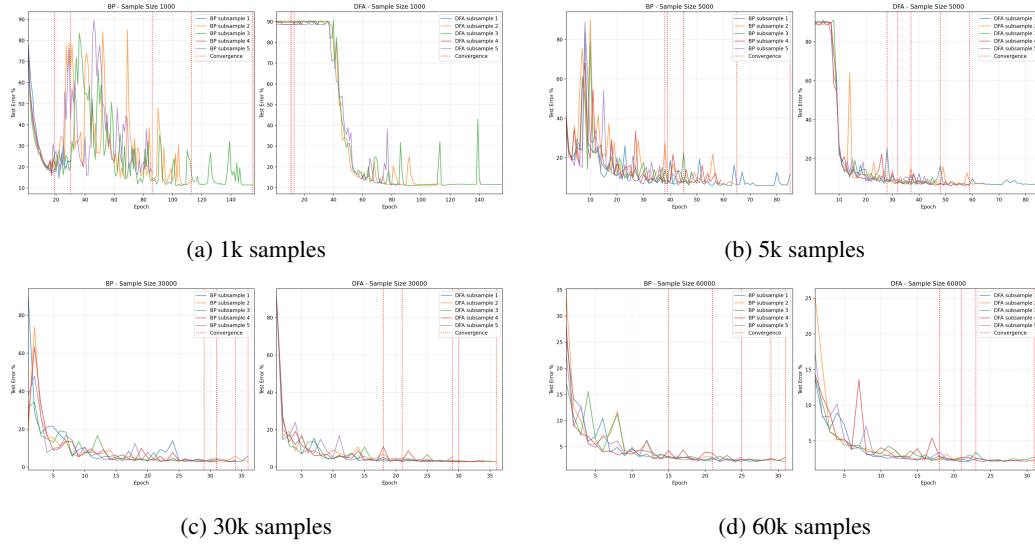


Figure 7: Error curves for BP (left) and DFA (right) at varying dataset sizes. a) Dataset of 1k samples b) Dataset of 5k samples c) Dataset of 30k samples d) Dataset of 60k samples (full MNIST training set)

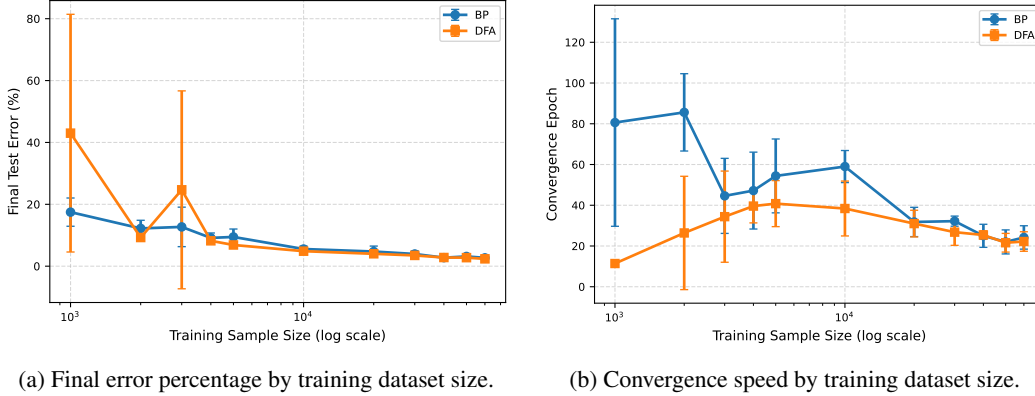


Figure 8: a) Final test loss of the 10x800 tanh network based on different training dataset sizes. a) Convergence speed of the 10x800 tanh network based on different training dataset sizes.

4 Discussion

This work provides an empirical comparison between the Direct Feedback Alignment algorithm and Backpropagation in feedforward networks trained on MNIST. This work builds upon Nøkland (2016), reproducing and extending the original paper to investigate convergence behavior and test performance across varying network architectures and data regimes. Across all architectures and data regimes considered, DFA matched BP in terms of final test performance and, in many cases, exhibited matching convergence speeds. In shallow and moderately deep networks (up to 10 hidden layers), DFA achieved test errors that were within a small margin of BP, consistent with the original findings of Nøkland (2016).

The 30x800 tanh network highlights a qualitative difference between the two algorithms. For deep networks with saturating activation functions such as tanh, BP exhibits the classic vanishing gradient problem where gradient norms decay rapidly through layers, and training stalls at suboptimal error levels [9]. DFA, on the other hand, maintains stable gradient norms throughout deep networks due to the direct feedback mechanism, avoiding the recursive nature of BP. In our experiments, this property allowed DFA to drastically outperform BP in deep networks, which is a strong indication that DFA is a promising candidate for biologically plausible deep learning in the presence of saturating nonlinearities.

The convergence analysis across depths and widths shows that, within the range investigated, convergence speed is only weakly correlated with network architecture for both BP and DFA. Further studies in broader ranges could potentially reveal bigger picture trends, but are outside the scope of this paper. The data-efficiency studies revealed that DFA is as sample efficient as BP. Both methods reached near-optimal performance with only one third of the MNIST training set compared to the full training set, and DFA did not require more data to approach its best test error. Notably, with small sample sizes, DFA showed a two-phase training behavior, with an initial plateau followed by sharp improvements in test error once alignment between forward and backward pathways had developed.

At the same time, this study has important limitations. All experiments were conducted on feedforward networks with tanh activation on a single, simple dataset. Hyperparameters were tuned coarsely and were fixed throughout the various experiments and thus may not reflect the best possible configurations for each experiment. In Nøkland (2016), DFA was observed to underperform BP on convolutional networks and on more challenging benchmarks. Understanding why DFA appears more competitive in fully connected networks than in convolutional ones, and devising subsequent variations to close this gap are important future directions to investigate following this study.

Overall, the results of this empirical study strengthen the case of DFA as a suitable alternative to BP in deep, fully connected networks. DFA eliminates the weight transport problem, uses locally available signals, and allows for parallel layer updates, yet still learns in a manner that matches BP’s competitiveness across a range of architectures and dataset sizes. These properties make DFA an attractive starting point for developing more biologically plausible learning rules.

5 Conclusion

This study empirically compared Direct Feedback Alignment and Backpropagation in deep, fully connected networks on MNIST. Across a range of depths, widths and data regimes, DFA matched BP in terms of final test performance and showed similar convergence behavior, while maintaining robustness of gradient norms in deep networks where BP suffers from vanishing gradients.

These results support DFA as a strong candidate for biologically plausible learning: it avoids weight transport, uses only local information, allows for parallel update of layers, and still maintains much of BP’s effectiveness. More broadly, these findings add to growing evidence that learning rules respecting biological constraints need not sacrifice performance, and suggest that further refinement of DFA-style mechanisms could bring machine learning and neuroscience closer together.

6 Code Availability

The code for this study is available at <https://github.com/allietje/apmth226>.

Acknowledgments and Disclosure of Funding

The author would like to thank Professor Cengiz Pehlevan and the course staff for providing guidance and advice for the project proposal and the project itself. No funding to disclose.

References

- [1] Rumelhart, D.E., Hinton, G.E. & Williams, R.J. (1986) Learning representations by back-propagating errors. *Nature* **323**:533–536.
- [2] Nøklund, A. (2016) Direct feedback alignment provides learning in deep neural networks. *Advances in Neural Information Processing Systems* **29**.
- [3] Refinetti, M., d’Ascoli, S., Ohana, R. & Goldt, S. (2021) The dynamics of learning with feedback alignment. *International Conference on Machine Learning*.
- [4] Liao, Q., Leibo, J.Z. & Poggio, T. (2016) How important is weight symmetry in backpropagation? *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [5] Whittington, J.C.R. & Bogacz, R. (2019) Theories of error back-propagation in the brain. *Trends in Cognitive Sciences* **23**(3):235–250.
- [6] Grossberg, S. (1987) Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science* **11**(1):23–63.
- [7] Pascanu, R., Mikolov, T. & Bengio, Y. (2013) On the difficulty of training Recurrent Neural Networks. *arXiv preprint arXiv:1211.5063*.
- [8] van der Maaten, L.J.P. & Hinton, G.E. (2008) Visualizing high-dimensional data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.
- [9] Hochreiter, S. (1998) The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2), 107–116.

A Technical Appendices and Supplementary Material

Direct Feedback Alignment (DFA) Procedure

The training step for DFA proceeds as follows:

1. **Forward pass:** Compute activations layer-by-layer.

$$h^0 = x, \quad a^l = W_l h^{l-1}, \quad h^l = \tanh(a^l) \quad \text{for } l = 1, \dots, L$$

Final output layer:

$$\hat{y} = \sigma(W_L h^{L-1})$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ is the sigmoid function.

2. **Compute output error:**

$$e = \hat{y} - y$$

3. **Update top layer using standard backpropagation:**

$$\Delta W_L = e (h^{L-1})^\top$$

4. **Update hidden layers using direct feedback:**

for $l = L - 1$ down to 1 :

$$\delta a^l = (B_l e) \odot (1 - \tanh^2(a^l)), \quad \Delta W_l = \delta a^l (h^{l-1})^\top$$

where B_l are fixed random matrices drawn from $\mathcal{N}(0, 1/\sqrt{n_{\text{in}}})$.

5. Apply weight updates using the chosen optimizer (RMSprop in the experiments in accordance to Nøklund (2016)).